Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №15

Дисциплина: Основы программирования на Python

Тема: Разработка веб-приложения на Python с использованием Flask

Вариант 2: Библиотека

Цель работы

Получить практические навыки разработки веб-приложений на Python с использованием микрофреймворка Flask. В ходе работы необходимо изучить архитектуру Flask-приложения, научиться создавать маршруты и обработчики HTTP-запросов, освоить работу с параметрами URL и HTTP-запросов, научиться формировать HTML-страницы и JSON-ответы, реализовать CRUD-операции, использовать шаблоны Jinja2, организовать обработку ошибок, подключить реляционную базу данных, применить SQLAlchemy для работы с данными, разделить веб-слой, бизнес-логику и доступ к данным и реализовать приложение с серверным рендерингом.

Постановка индивидуального задания (Вариант 2. Библиотека)

Разработать веб-приложение библиотеки с сущностями Book (книга), Author (автор), Reader (читатель) и Loan (выдача). Реализовать каталог книг с поиском, фильтрацией и сортировкой, просмотр карточки книги, добавление, редактирование и удаление книг, выдачу книги читателю и возврат книги, справочник читателей с историей выдач и страницу статистики. Приложение должно содержать серверный рендеринг страниц и параллельно предоставлять REST API для тех же операций.

Краткие теоретические сведения

Flask — микрофреймворк, который предоставляет минимальный набор средств: маршрутизацию, обработку запросов и ответов, работу с шаблонами и сессиями. Маршрут объявляется декоратором app.route или route объекта Blueprint с указанием пути и допустимых методов. Параметры пути передаются в функцию обработчика как аргументы, а параметры строки запроса доступны через словарь request.args. Тело JSON-запроса извлекается методом request.get_json(), данные HTML-форм — через request.form.

Blueprint — механизм группировки маршрутов, позволяющий разделить приложение на модули. В работе используются два blueprint: web с префиксом корневого пути для HTML-страниц и api с префиксом /api для JSON-интерфейса. Такое разделение позволяет одному приложению обслуживать и пользователей, и программных клиентов.

Шаблоны Jinja2 позволяют отделить разметку от логики. Наследование через конструкцию extends и блоки block обеспечивает единый каркас страниц, а конструкции for и if формируют списки и условные блоки. Автоматическое экранирование вывода защищает от внедрения HTML-кода, а механизм flash выводит пользователю одноразовые сообщения после перенаправления.

Доступ к данным реализован через SQLAlchemy: модели описывают таблицы, связи задаются через relationship, а запросы выполняются через объект Session. Слой сервиса содержит бизнес-логику и валидацию, а веб-слой и API вызывают методы сервиса, не работая с базой напрямую.

Структура проекта

Компонент	Назначение
app/__init__.py	Фабрика create_app(): регистрация blueprint, обработчики ошибок, журналирование запросов, заполнение базы
app/config.py	Настройки приложения: строка подключения, секретный ключ, режим отладки
app/database.py	Базовый класс Base, создание движка, включение внешних ключей, фабрика сессий

Компонент	Назначение
app/models/models.py	ORM-модели Author, Reader, Book, Loan со связями
app/repositories/repositories.py	Репозитории книг, авторов, читателей и выдач
app/services/services.py	LibraryService: валидация, CRUD, выдача и возврат, статистика
app/routes/web.py	Маршруты HTML-интерфейса с серверным рендерингом
app/routes/api.py	Маршруты REST API с префиксом /api
app/templates/	Десять шаблонов Jinja2: каркас, каталог, карточка, форма, выдача, читатели, страницы ошибок
tests/	Тесты сервисного слоя и тесты страниц и API

Модель данных

Таблица	Поля	Связи
authors	id, full_name, country	Один автор — много книг
readers	id, full_name, email, phone, registered_at	Один читатель — много выдач
books	id, title, author_id, genre, year, isbn, price, copies_total, copies_available	Ссылка на автора, много выдач
loans	id, book_id, reader_id, loan_date, due_date, return_date	Ссылки на книгу и читателя

Связь книги с автором задана внешним ключом author_id, а выдачи связывают книгу и читателя, реализуя отношение «многие ко многим» с дополнительными атрибутами: датой выдачи, сроком возврата и фактической датой возврата. Такая структура позволяет получить историю выдач как по книге, так и по читателю.

Маршруты веб-интерфейса

Маршрут	Методы	Назначение
/	GET	Главная страница со статистикой и последними поступлениями
/books	GET	Каталог с поиском, фильтром по жанру, наличию и сортировкой
/books/<id>	GET	Карточка книги с историей выдач
/books/create	GET, POST	Форма добавления книги
/books/<id>/edit	GET, POST	Форма редактирования книги
/books/<id>/delete	POST	Удаление книги с перенаправлением в каталог
/books/<id>/borrow	GET, POST	Выдача книги читателю с выбором срока
/loans/<id>/return	POST	Возврат книги

Маршрут	Методы	Назначение
/readers	GET	Список читателей
/readers/<id>	GET	Карточка читателя с активными выдачами

Изменяющие операции выполняются методом POST с последующим перенаправлением по принципу Post/Redirect/Get, что предотвращает повторную отправку формы при обновлении страницы. Результат операции сообщается пользователю через flash-сообщение с категорией success или error.

Листинг 15.1 — Маршрут каталога с фильтрацией (app/routes/web.py)

```
@web.route("/books")
def books():
    library = service()
    search = request.args.get("search", "").strip()
    genre = request.args.get("genre") or None
    sort = request.args.get("sort", "title")
    order = request.args.get("order", "asc")
    available = request.args.get("available") == "1"


    items = library.list_books(
        search=search or None,
        genre=genre,
        available_only=available,
        sort=sort,
        order=order,
    )
    return render_template(
        "list.html", books=items, genres=library.genres(),
        search=search, genre=genre, sort=sort,
        order=order, available=available)
```

REST API приложения

Второй blueprint предоставляет те же возможности в формате JSON, что позволяет использовать приложение из внешних клиентов и автоматических тестов.

Метод и путь	Параметры	Результат
GET /api/books	search, genre, available, sort, order, limit, offset	200, объект с total, limit, offset и items
GET /api/books/<id>	идентификатор	200 — книга, 404 — не найдена
POST /api/books	тело JSON	201 — созданная книга, 400 — ошибка валидации, 409 — конфликт
PUT /api/books/<id>	тело JSON	200 — заменённая книга
PATCH /api/books/<id>	тело JSON с частью полей	200 — изменённая книга
DELETE /api/books/<id>	идентификатор	204 без тела

Метод и путь	Параметры	Результат
GET /api/books/search	q (обязательный)	200, объект с query, count и items
GET /api/statistics	нет	200, статистика книг, читателей и выдач

Метод PATCH реализован через объединение текущих значений книги с переданными полями: клиент может изменить только один атрибут, не передавая остальные. Ограничение limit не позволяет запросить более ста записей за один раз, а offset отрицательных значений не принимает.

Шаблоны и обработка ошибок

Шаблоны построены на наследовании от base.html, который содержит каркас страницы, навигацию и блок вывода flash-сообщений. Каталог list.html выводит карточки книг и форму фильтрации, detail.html показывает сведения о книге и историю выдач, form.html используется и для создания, и для редактирования, изменяя заголовок и адрес отправки в зависимости от режима. Отдельные шаблоны 404.html и error.html отображают страницы ошибок.

Обработчики ошибок зарегистрированы на уровне приложения и различают два типа клиентов: для путей, начинающихся с /api, возвращается JSON с полем detail и машинным кодом, для остальных путей отображается HTML-страница. Ошибки ValidationError дают код 400, NotFoundError — 404, ConflictError — 409, а непредвиденные исключения — 500 с записью в журнал. Функции before_request и after_request замеряют и журналируют время обработки каждого запроса.

Листинг 15.2 — Обработчики ошибок (app/__init__.py)

```
def is_api() -> bool:
    return request.path.startswith("/api")


@app.errorhandler(ValidationError)
def _validation(error):
    if is_api():
        return jsonify({"detail": str(error), "code": error.code}), 400
    return render_template("error.html", message=str(error), code=400), 400


@app.errorhandler(NotFoundError)
def _not_found(error):
    if is_api():
        return jsonify({"detail": str(error), "code": error.code}), 404
```

```
    return render_template("404.html", message=str(error)), 404


@app.errorhandler(ConflictError)
def _conflict(error):
    if is_api():
        return jsonify({"detail": str(error), "code": error.code}), 409
    return render_template("error.html", message=str(error), code=409), 409
```

Раздел тестирования

Тесты разделены на два файла: тесты сервисного слоя проверяют бизнес-правила, тесты приложения — HTML-страницы и REST API. Каждый тест работает с временной базой данных и создаёт приложение через фабрику create_app с тестовой конфигурацией.

Файл тестов	Кол-во	Что проверяется
tests/test_service.py	14	Создание, получение, изменение и удаление книги, поиск, список жанров, отсутствие обязательного поля, некорректный ISBN, отсутствующая книга, дубликат ISBN, выдача отсутствующему читателю, выдача без экземпляров, возврат книги, статистика
tests/test_api.py	21	Главная страница, каталог с поиском, фильтром и сортировкой, карточка книги и страница 404, форма создания, страница читателей, статистика, а также все методы REST API и обработка ошибок

Все тридцать пять тестов проходят успешно. Дополнительно приложение проверялось вручную в браузере: выполнялся поиск по названию, применялся фильтр по жанру и наличию экземпляров, открывалась карточка книги, создавалась новая книга через форму, оформлялась выдача читателю, затем возврат книги со страницы читателя и удаление записи из каталога. Отдельно проверялись JSON-запросы к API и корректность страниц ошибок.

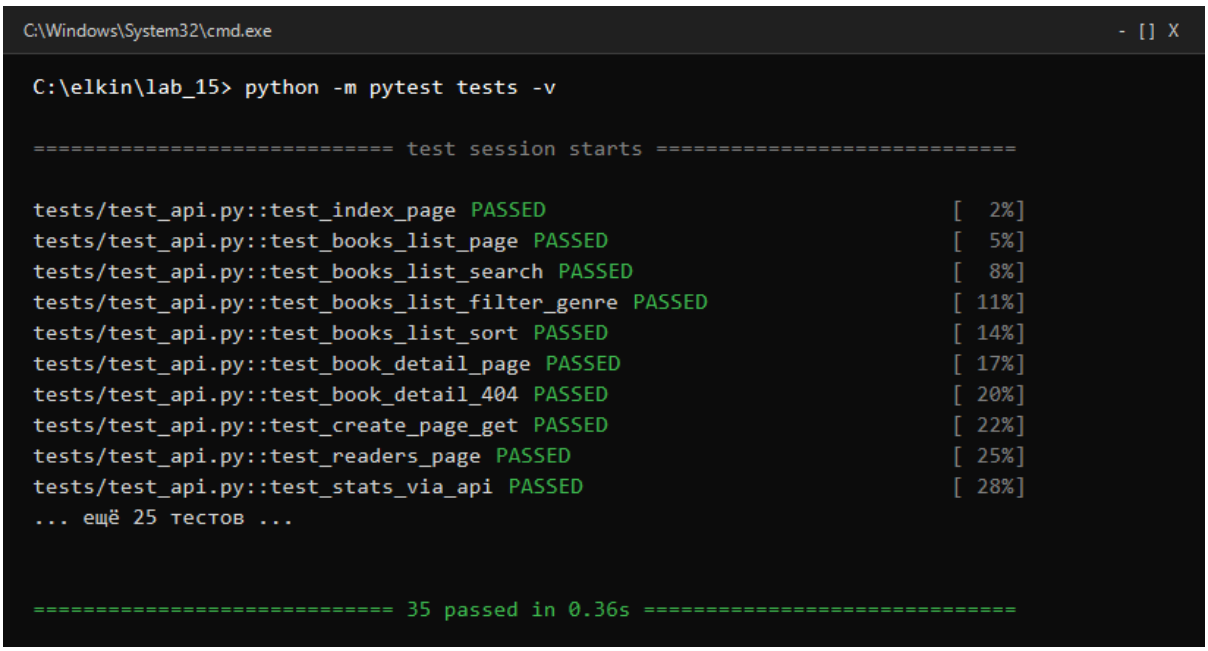


Рисунок 15.1 — Результат прохождения тестов страниц и REST API Flask-приложения

Листинги ключевых фрагментов программы

Листинг 15.3 — Фабрика приложения (app/__init__.py)

```
def create_app(config: dict | None = None,
               with_seed: bool = True) -> Flask:
    app = Flask(__name__)
    app.config.update(SECRET_KEY=settings.secret_key)
    if config:
        app.config.update(config)


    app.register_blueprint(web)
    app.register_blueprint(api)


    @app.before_request
    def _start_timer():
        g.started = time.perf_counter()


    @app.after_request
    def _log_request(response):
        elapsed = (time.perf_counter()
                   - getattr(g, "started", time.perf_counter())) * 1000
        logger.info("%s %s -> %s (%.1f ms)", request.method,
                    request.path, response.status_code, elapsed)
        return response


    if with_seed:
        create_tables()
        session = get_session()
        seed_database(session)
        session.close()


    return app
```

Листинг 15.4 — Создание книги через API (app/routes/api.py)

```
@api.post("/books")
def create_book():
    payload = request.get_json(silent=True)
    if not isinstance(payload, dict):
        raise ValidationError("Тело запроса должно быть JSON-объектом")
    book = service().create_book(payload)
    return jsonify(book_to_dict(book)), 201


@api.delete("/books/<int:book_id>")
def delete_book(book_id: int):
    service().delete_book(book_id)
    return "", 204
```

Листинг 15.5 — Выдача книги через форму (app/routes/web.py)

```
if request.method == "POST":
    try:
        library.issue_book(
            book_id,
            int(request.form.get("reader_id", 0)),
            int(request.form.get("days", 14)),
        )
        flash("Книга выдана читателю", "success")
        return redirect(url_for("web.book_detail", book_id=book_id))
    except (ValidationError, NotFoundError) as error:
        flash(str(error), "error")

return render_template("borrow.html", book=book,
                       readers=library.readers.list())
```

Вывод

В ходе выполнения лабораторной работы разработано веб-приложение библиотеки на микрофреймворке Flask. Реализованы серверный рендеринг HTML-страниц с шаблонами Jinja2 и параллельный REST API, использующий то же ядро бизнес-логики. Освоены маршрутизация с параметрами пути и строки запроса, работа с формами и JSON-телом, перенаправления и flash-сообщения, а также единая обработка ошибок для HTML- и JSON-клиентов. Данные хранятся в реляционной базе через SQLAlchemy, а разделение на модели, репозитории, сервисы и маршруты обеспечивает сопровождение и тестирование приложения, что подтверждают тридцать пять автоматических тестов.